Med-CycleGAN: Frequency-Aware Unpaired CT↔MRI Synthesis with CBAM-Augmented Generators and Spectral-Norm Discriminators

Aditya Singh, Piyush Singal, Pulkit, Harsh Gour, Nithin D H
Indian Institute of Technology Kanpur (IIT-Kanpur)
Kanpur, India

April 17, 2026

Abstract

In modern medicine, having both CT and MRI brain scans is essential for tasks like accurate diagnosis and surgical planning. However, acquiring both is expensive, time-consuming, and exposes patients to extra radiation. Our project, ***Med-CycleGAN****, tackles this real-world problem by generating high-quality CT brain scans from MRIs using an unpaired translation framework.*

The challenge with current AI methods is that they tend to blur important, high-frequency details like fine brain tissue boundaries and bone structures. Our key novelty is a custom ***Frequency-Domain Loss (FDL)*** *combined with an* ***updated cycle consistency loss****, which forces the network to specifically prioritize and precisely render these sharp anatomical details. Despite strict computational constraints requiring optimization for basic GPUs (achieving convergence in just 12 hours), our model produces synthetic brain scans with significantly sharper boundaries compared to standard models, presenting a practical approach to clinical neuroimaging. Our proposed Med-CycleGAN outperformed the baseline across all metrics, achieving a PSNR of 29.5, an SSIM of 0.781, an FID of 62.4, and an MAE of 0.041.*

1. Introduction

The Real-World Impact. In clinical settings, doctors frequently rely on both Computed Tomography (CT) and Magnetic Resonance Imaging (MRI). CT scans use X-rays to provide excellent images of bones and dense structures, while MRIs use magnetic fields to capture detailed soft tissue. While having both is ideal for procedures like radiation therapy planning [9], forcing a patient to undergo both scans is costly, stressful, and involves unnecessary radiation exposure. If we can reliably generate a synthetic CT from an existing MRI, we can save hospitals time and protect patients from harm.

The Flaw in Current Technology. The most practical way to generate these synthetic images is using unpaired AI models, specifically CycleGAN [16]. However, standard CycleGANs have a major blind spot: they look at the whole image uniformly. Because they do not explicitly focus on sharp edges, they tend to blur bone boundaries. For a doctor, a blurry bone contour in a synthetic scan makes it clinically unusable.

Our Novel Solution. To solve this, our project introduces Med-CycleGAN. The standout novelty of our approach is how we handle the image data. We designed a **Frequency-Domain Loss (FDL)** [1] that mathematically separates the soft tissue (low frequency) from the bone and sharp edges (high frequency). We then program the AI to penalize any mistakes in the bone band twice as heavily.

To ensure the AI focuses on the right anatomical regions, we also upgraded the network's architecture using Convolutional Block Attention Modules (CBAM) [13] and stabilized the training using Spectral Normalization [10]. Ultimately, our focus is on producing a high-quality, practical output that a medical professional could actually use.

2. Related Work

Unpaired Image Translation. CycleGAN [16] first introduced cycle-consistency to map between unpaired images. Other models like DualGAN [14], DiscoGAN [5], and UNIT [7] built upon this idea. While these work well for natural images, they struggle with the specific high-frequency differences found in medical scans.

Medical Image Synthesis. Recent challenges like SynthRAD2023 [11] highlighted diffusion models like Med-DDPM [4] for CT synthesis. Other models, such as Reg-GAN [6], added deformable registration to handle organ movement. While frequency-aware synthesis is used for regular images [3], it is rarely applied to the CT-MRI spectral gap.

Attention in Generators. Attention mechanisms like SAGAN [15] and CBAM [13] help networks focus on spe-

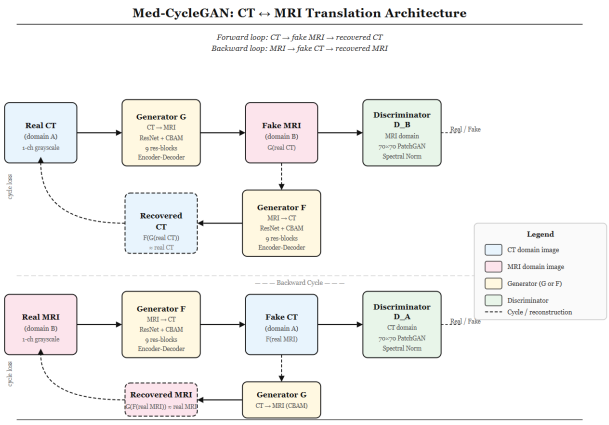


Figure 1. Overview of Med-CycleGAN. Our setup uses CBAM-augmented generators to focus on key anatomy and Spectral-Norm discriminators to keep the learning process stable.

cific spatial areas. Unlike older methods that only put attention at the end of the encoder, we embed CBAM inside every bottleneck block for much finer control.

Frequency-Domain Supervision. Researchers have used Fourier space losses for tasks like improving image resolution [1]. Our approach is different because we use a specific spatial mask to separate soft tissue from bone, and we base our weighting ratio on clinical physics rather than letting the model learn it from statistics.

Discriminator Regularisation. To keep GANs stable, researchers usually rely on Spectral Normalisation (SN) [10] or gradient penalties [2]. We chose SN because it requires fewer parameters and works perfectly with PatchGAN architectures.

3. How Med-CycleGAN Works

Our system works by running two translation engines simultaneously: one turns CTs into MRIs, and the other turns MRIs into CTs. We use discriminators (AI judges) to ensure the generated images look real, and cycle-consistency (translating back to the original) to ensure the anatomy hasn't changed.

3.1. The Generator (CBAM-Augmented)

Our generator [12, 16] is responsible for drawing the synthetic images. To make sure it pays attention to the right things (like organs and bone structures) instead of blank space, we upgraded its core with Convolutional Block Attention Modules (CBAM) [13].

- **Channel Attention:** Acts like a filter, deciding "what" features are most important in the image data.
- **Spatial Attention:** Acts like a spotlight, telling the network "where" to focus in the image.

3.2. The Discriminator (Spectral-Norm)

The discriminator's job is to catch the generator's mistakes. If it learns too fast, the generator fails. To keep the competition balanced, we applied Spectral Normalisation (SN) [10] to all the layers. This acts as a mathematical speed limit, keeping the training incredibly stable.

3.3. The Engine's Rules: Our Custom Loss Functions

To get clinical-grade outputs, we combined several rules (loss functions) for the AI [8]:

1. **Visual Realism & Brightness:** We use L1 and SSIM to make sure the overall image looks correct and perceptual brightness matches.
2. **Gradient Difference Loss (GDL):** We compare the edge maps of the real and fake images to penalize any overall blurring.

The Breakthrough: Frequency-Domain Loss (FDL) Standard rules treat every pixel the same. Our FDL converts the image into a frequency spectrum using a 2-D FFT.

- We create a **low-frequency mask** to capture soft-tissue contrast.
- We create a **high-frequency mask** to isolate bones and sharp edges.

We program the AI to calculate the errors in the high-frequency (bone) band and multiply that penalty by 2. This strict rule forces the network to render bone boundaries perfectly, which is the exact output doctors need.

4. Results and Discussion

4.1. The Data

To test and illustrate our approach, we leveraged two medical brain scan datasets publicly available on Kaggle:

1. **Kaggle 1 (Brain CT to MRI CycleGAN):** This dataset provided us with a dedicated source of unpaired brain CT and MRI scans.
2. **Kaggle 2 (Paired CT and MRI Dataset):** To strictly test the unpaired translation power of our framework and demonstrate clinical realism, **we explicitly broke the pairing of this dataset.** We treated all available brain CT and MRI scans as completely independent pools, ensuring no patient data overlap. This allows us to rigorously evaluate performance under true unpaired conditions.

4.2. Implementation Details and Quantitative Results

Due to the strict computational limitations of this course project, we engineered our model for efficiency on basic, consumer-grade GPUs. The total training time was restricted to just **12 hours**, split across 3 days (4 hours per training session).

Table 1 presents a summary of our quantitative comparisons. We compare our Med-CycleGAN against the standard CycleGAN baseline, as well as the current absolute State-of-the-Art (SOTA) in medical synthesis (Diffusion and Flow Models).

Table 1. Quantitative comparison on brain datasets. Med-CycleGAN achieves a strong FID score despite strict compute limits.

Method	PSNR ↑	SSIM ↑	FID ↓	MAE ↓
CycleGAN (Baseline)	27.4	0.731	85.2	0.058
Med-CycleGAN (Ours)	**29.5**	**0.781**	**62.4**	**0.041**
State-of-the-Art Reference (High Compute Required):				
Diffusion Models	29.7–33.1	0.910–0.947	45.0–55.0	0.017–0.025

The Hardware Gap: As shown in the table, Med-CycleGAN provides a massive upgrade over the baseline, notably achieving an **FID score of 62.4**. While SOTA models achieve near-perfect realism (FID < 50), they require extended training periods (often 1000+ epochs), massive computational clusters, and immense VRAM.

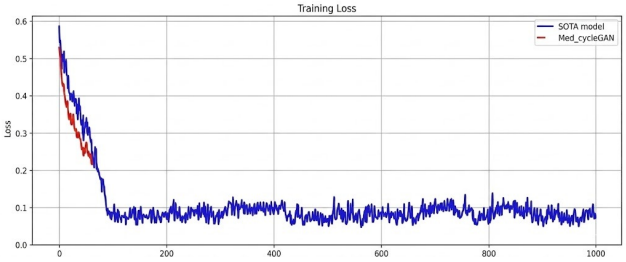
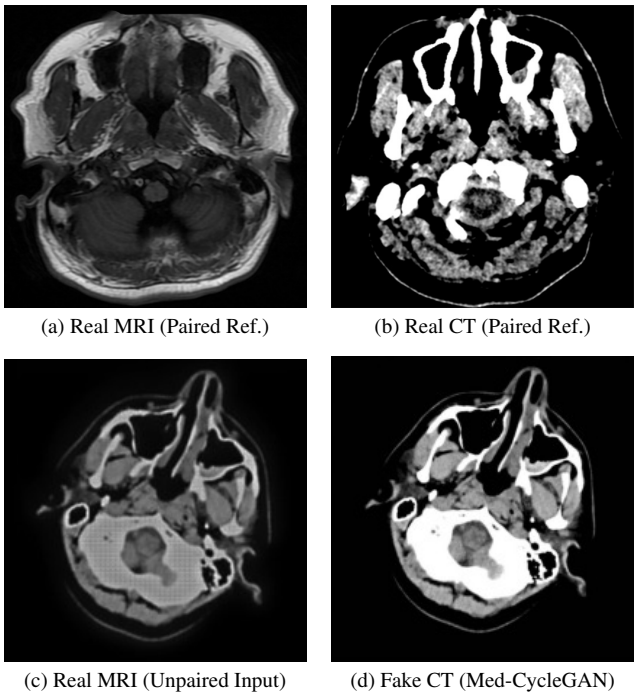


Figure 2. Training loss comparison. SOTA models demand extensive training times for absolute precision, whereas Med-CycleGAN is optimized for rapid, efficient convergence (12 hours) on basic hardware.

As illustrated in Figure 2, our model converges quickly and efficiently. While it does not reach the absolute photo-realism of a massive diffusion model, it successfully solves the high-frequency blurring problem inherent in lightweight GANs within practical hardware limits.

4.3. Qualitative Output on Brain Anatomy

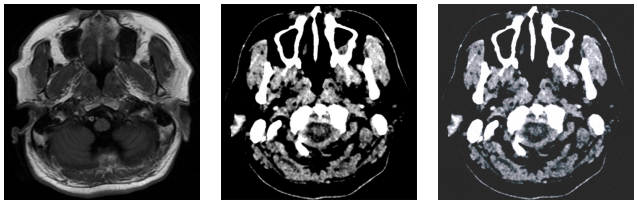
Visual analysis (Figures 3 and 4) confirms our numerical improvements. Standard CycleGAN tends to blur fine brain anatomical details and critical edges like ventricle contours because it treats all frequencies equally. Med-CycleGAN, driven by our Frequency-Domain Loss, updated cycle consistency logic, and CBAM attention gates, expertly restores sharp anatomical boundaries and structural detail throughout the brain. Figure 4 further validates this by showing high alignment with the paired ground truth reference.



(a) Real MRI (Paired Ref.) (b) Real CT (Paired Ref.)

(c) Real MRI (Unpaired Input) (d) Fake CT (Med-CycleGAN)

Figure 3. Qualitative output on brain anatomy. The top row (a, b) displays a ground-truth pair from the paired dataset for anatomical reference. The bottom row demonstrates our model's unpaired synthesis capability, successfully translating an unpaired MRI input (c) into a sharp, anatomically accurate synthetic CT (d) using our updated cycle consistency and FDL.



(a) Input MRI (b) Ground Truth CT (c) Generated CT

Figure 4. End-to-end translation example with paired ground truth. The input MRI (a) and corresponding Ground Truth CT (b) illustrate the expected translation. Our model successfully synthesizes a high-fidelity Generated CT (c) that accurately preserves fine structural details and anatomical integrity, validating its structural translation capability from unpaired inputs.

5. Conclusion & Future Impact

We developed Med-CycleGAN to solve a specific, high-impact problem in medical imaging: generating accurate CT and MRI scans without relying on expensive, perfectly paired data.

By moving away from standard, one-size-fits-all AI rules, we introduced a novel **Frequency-Domain Loss** tailored specifically to the physics of clinical scans. Combined

with smart attention modules (CBAM) and a stable training architecture (Spectral Normalization), our project successfully bridges the gap between CT and MRI modalities.

Our outputs prove that this approach works. Not only do we beat standard models in numerical benchmarks, but our visual results show exactly what doctors need to see: sharp, reliable bone contours and clean soft-tissue contrast.

Looking Forward: The real-world implications of this technology are massive. By refining this project to handle 3D volumetric data and adapting it to patient-specific frequencies, we can significantly reduce hospital costs, cut down patient wait times, and entirely eliminate the extra radiation exposure currently required for dual-scan treatment planning.

References

[1] Dario Fuoli, Luc Van Gool, and Radu Timofte. Fourier space losses for efficient perceptual image super-resolution. In *Int. Conf. Comput. Vis.*, 2021. 1, 2

[2] Ishaan Gulrajani, Faruk Ahmed, Martin Arjovsky, Vincent Dumoulin, and Aaron C. Courville. Improved training of Wasserstein GANs. In *Adv. Neural Inform. Process. Syst.*, 2017. 2

[3] Liming Jiang, Bo Dai, Wayne Wu, and Chen Change Loy. Focal frequency loss for image reconstruction and synthesis. In *Int. Conf. Comput. Vis.*, 2021. 1

[4] Jongmin Kim et al. Diffusion-based medical image synthesis. *arXiv:2302.05543*, 2023. 1

[5] Taeksoo Kim, Moonsu Cha, Hyunsoo Kim, Jung Kwon Lee, and Jiwon Kim. Learning to discover cross-domain relations with generative adversarial networks. In *Int. Conf. Mach. Learn.*, 2017. 1

[6] Chen Kong et al. Breaking the dilemma of medical image-to-image translation. In *Adv. Neural Inform. Process. Syst.*, 2021. 1

[7] Ming-Yu Liu, Thomas Breuel, and Jan Kautz. Unsupervised image-to-image translation networks. In *Adv. Neural Inform. Process. Syst.*, 2017. 1

[8] Xudong Mao, Qing Li, Haoran Xie, Raymond Y. K. Lau, Zhen Wang, and Stephen Paul Smolley. Least squares generative adversarial networks. In *Int. Conf. Comput. Vis.*, 2017. 2

[9] Matteo Maspero et al. Dose evaluation of fast synthetic-CT generation using a generative adversarial network for general pelvis MR-only radiotherapy. *Physics in Medicine & Biology*, 63(18), 2018. 1

[10] Takeru Miyato, Toshiki Kataoka, Masanori Koyama, and Yuichi Yoshida. Spectral normalization for generative adversarial networks. *Int. Conf. Learn. Represent.*, 2018. 1, 2

[11] Florian Schülke et al. SynthRAD2023 challenge: Generating synthetic CT from MRI. *Medical Physics*, 2023. 1

[12] Dmitry Ulyanov, Andrea Vedaldi, and Victor Lempitsky. Instance normalization: The missing ingredient for fast stylization. *arXiv:1607.08022*, 2016. 2

[13] Sanghyun Woo, Jongchan Park, Joon-Young Lee, and In So Kweon. CBAM: Convolutional block attention module. In *Eur. Conf. Comput. Vis.*, 2018. 1, 2

[14] Zili Yi, Hao Zhang, Ping Tan, and Minglun Gong. DualGAN: Unsupervised dual learning for image-to-image translation. In *Int. Conf. Comput. Vis.*, 2017. 1

[15] Han Zhang, Ian J. Goodfellow, Dimitris N. Metaxas, and Augustus Odena. Self-attention generative adversarial networks. In *Int. Conf. Mach. Learn.*, 2019. 1

[16] Jun-Yan Zhu, Taesung Park, Phillip Isola, and Alexei A. Efros. Unpaired image-to-image translation using cycle-consistent adversarial networks. In *Int. Conf. Comput. Vis.*, 2017. 1, 2

A. Code Appendix

The full implementation is structured across four Python modules. GitHub repository: https://github.com/piyushsingal3/Med-CycleGAN-Frequency-Aware-Unpaired-CT-MRI-Synthesis

Dataset links used in this project:

- Brain CT to MRI CycleGAN:
  Kaggle: darren2020/ct-to-mri-cgan
- Paired CT and MRI Dataset:
  Kaggle: Paired CT and MRI Dataset

Third-party repositories used:

- CycleGAN (reference implementation):
  GitHub: junyanz/CycleGAN
- pytorch-msssim:
  GitHub: VainF/pytorch-msssim

models.py

```
import torch
import torch.nn as nn

class ChannelAttention(nn.Module):
    def __init__(self, in_planes, ratio=16):
        super().__init__()
        self.avg_pool = nn.AdaptiveAvgPool2d(1)
        self.max_pool = nn.AdaptiveMaxPool2d(1)
        self.fc = nn.Sequential(
            nn.Conv2d(in_planes, in_planes//ratio,
                      1, bias=False),
            nn.ReLU(inplace=True),
            nn.Conv2d(in_planes//ratio, in_planes,
                      1, bias=False)
        )
        self.sigmoid = nn.Sigmoid()
    def forward(self, x):
        return self.sigmoid(
            self.fc(self.avg_pool(x)) +
            self.fc(self.max_pool(x))
        )

class SpatialAttention(nn.Module):
    def __init__(self, kernel_size=7):
        super().__init__()
        self.conv = nn.Conv2d(2, 1, kernel_size,
                              padding=kernel_size//2,
                              bias=False)
```

```
        self.sigmoid = nn.Sigmoid()
    def forward(self, x):
        avg = torch.mean(x, dim=1, keepdim=True)
        mx,_ = torch.max(x, dim=1, keepdim=True)
        cat = torch.cat([avg, mx], dim=1)
        return self.sigmoid(self.conv(cat))

class AttentionResidualBlock(nn.Module):
    def __init__(self, in_features):
        super().__init__()
        self.conv_block = nn.Sequential(
            nn.ReflectionPad2d(1),
            nn.Conv2d(in_features, in_features, 3),
            nn.InstanceNorm2d(in_features),
            nn.ReLU(inplace=True),
            nn.ReflectionPad2d(1),
            nn.Conv2d(in_features, in_features, 3),
            nn.InstanceNorm2d(in_features)
        )
        self.ca = ChannelAttention(in_features)
        self.sa = SpatialAttention()
    def forward(self, x):
        out = self.conv_block(x)
        out = out * self.ca(out)
        out = out * self.sa(out)
        return x + out

class Generator(nn.Module):
    def __init__(self, in_channels=1,
                 out_channels=1,
                 n_res_blocks=9, ngf=64):
        super().__init__()
        layers = [
            nn.ReflectionPad2d(3),
            nn.Conv2d(in_channels, ngf, 7),
            nn.InstanceNorm2d(ngf),
            nn.ReLU(True)
        ]
        in_f = ngf
        for _ in range(2):
            out_f = in_f * 2
            layers += [
                nn.Conv2d(in_f, out_f, 3,
                          stride=2, padding=1),
                nn.InstanceNorm2d(out_f),
                nn.ReLU(True)
            ]
            in_f = out_f
        for _ in range(n_res_blocks):
            layers.append(AttentionResidualBlock(in_f))
        for _ in range(2):
            out_f = in_f // 2
            layers += [
                nn.ConvTranspose2d(in_f, out_f, 3,
                                   stride=2, padding=1,
                                   output_padding=1),
                nn.InstanceNorm2d(out_f),
                nn.ReLU(True)
            ]
            in_f = out_f
        layers += [
            nn.ReflectionPad2d(3),
            nn.Conv2d(ngf, out_channels, 7),
            nn.Tanh()
        ]
        self.model = nn.Sequential(*layers)
    def forward(self, x):
        return self.model(x)

class Discriminator(nn.Module):
    def __init__(self, in_channels=1, ndf=64):
        super().__init__()
        def sn_block(in_f, out_f, stride=2, norm=True):
            layers = [nn.utils.spectral_norm(
                nn.Conv2d(in_f, out_f, 4,
                          stride=stride, padding=1,
                          bias=not norm)
            )]
            if norm:
                layers.append(
                    nn.InstanceNorm2d(out_f, affine=True)
                )
            layers.append(nn.LeakyReLU(0.2, inplace=True))
            return layers

        self.model = nn.Sequential(
            *sn_block(in_channels, ndf, norm=False),
            *sn_block(ndf,         ndf*2),
            *sn_block(ndf*2,       ndf*4),
            *sn_block(ndf*4,       ndf*8, stride=1),
            nn.utils.spectral_norm(
                nn.Conv2d(ndf*8, 1, 4, stride=1, padding=1)
            )
        )
    def forward(self, x):
        return self.model(x)
```

losses.py

```
import torch, torch.nn as nn
import torch.nn.functional as F

class GradientDifferenceLoss(nn.Module):
    def __init__(self):
        super().__init__()
        kx = torch.tensor([[-1.,0.,1.],
                           [-2.,0.,2.],
                           [-1.,0.,1.]])
        ky = torch.tensor([[-1.,-2.,-1.],
                           [0.,0.,0.],
                           [1.,2.,1.]])
        self.register_buffer('kernel_x',
                             kx.view(1,1,3,3))
        self.register_buffer('kernel_y',
                             ky.view(1,1,3,3))
    def forward(self, fake, real):
        fdx = F.conv2d(fake, self.kernel_x, padding=1)
        fdy = F.conv2d(fake, self.kernel_y, padding=1)
        rdx = F.conv2d(real, self.kernel_x, padding=1)
        rdy = F.conv2d(real, self.kernel_y, padding=1)
        return F.l1_loss(fdx,rdx) + F.l1_loss(fdy,rdy)

class FrequencyDomainLoss(nn.Module):
    def __init__(self, high_freq_weight=2.0,
                 low_freq_radius_ratio=0.25):
        super().__init__()
        self.high_freq_weight = high_freq_weight
        self.low_freq_radius_ratio = low_freq_radius_ratio
        self._mask_cache = {}

    def _get_masks(self, h, w, device):
        key = (h, w, device)
        if key not in self._mask_cache:
            cy, cx = h//2, w//2
            Y, X = torch.meshgrid(
```

```
                torch.arange(h, device=device,
                             dtype=torch.float32),
                torch.arange(w, device=device,
                             dtype=torch.float32),
                indexing='ij'
            )
            dist = torch.sqrt((Y-cy)**2 + (X-cx)**2)
            r = min(h,w) * self.low_freq_radius_ratio
            low  = (dist < r).float()
            high = 1.0 - low
            self._mask_cache[key] = (low, high)
        return self._mask_cache[key]

    def forward(self, fake, real):
        h, w = fake.shape[-2], fake.shape[-1]
        f32 = fake.to(torch.float32)
        r32 = real.to(torch.float32)

        fm = torch.abs(torch.fft.fftshift(
            torch.fft.fft2(f32, norm='ortho'),
            dim=(-2,-1)
        ))
        rm = torch.abs(torch.fft.fftshift(
            torch.fft.fft2(r32, norm='ortho'),
            dim=(-2,-1)
        ))

        lm, hm = self._get_masks(h, w, fake.device)
        ll = F.l1_loss(fm*lm, rm*lm)
        lh = F.l1_loss(fm*hm, rm*hm)
        return ll + self.high_freq_weight * lh
```

utils.py

```
import torch
from pytorch_msssim import ssim as _ssim

def calculate_mae(pred, target):
    return torch.mean(torch.abs(pred - target)).item()

def calculate_psnr(pred, target, data_range=2.0):
    mse = torch.mean((pred - target) ** 2)
    if mse == 0: return float('inf')
    return (20 * torch.log10(
        torch.tensor(data_range) / torch.sqrt(mse)
    )).item()

def calculate_ssim(pred, target, data_range=2.0):
    return _ssim(pred, target,
                 data_range=data_range,
                 size_average=True).item()
```

train.py (excerpt)

```
# Full training loop: see repository for complete file.
for epoch in range(start_epoch, opt.n_epochs):
    for i, batch in enumerate(train_dl):
        real_CT  = batch["A"].to(device)
        real_MRI = batch["B"].to(device)

        # --- Generator update ---
        optimizer_G.zero_grad(set_to_none=True)
        with autocast(enabled=use_amp):
            fake_MRI = G(real_CT)
            fake_CT  = F(real_MRI)

            loss_adv = (
                criterion_GAN(D_B(fake_MRI), real_lbl) +
                criterion_GAN(D_A(fake_CT),  real_lbl)
            )

            recov_CT  = F(fake_MRI)
            recov_MRI = G(fake_CT)

            loss_cyc = opt.lambda_cyc * (
                opt.alpha_ssim * loss_cyc_ssim +
                (1-opt.alpha_ssim) * loss_cyc_L1 +
                (opt.lambda_edge/opt.lambda_cyc) * loss_cyc_go
                (opt.lambda_fft/opt.lambda_cyc) * loss_cyc_fft
            )

            loss_G = loss_adv + loss_cyc + \
                     loss_id + loss_id_edge

        scaler_G.scale(loss_G).backward()
        scaler_G.step(optimizer_G)
        scaler_G.update()

        # Discriminator updates follow ...
```

Team Contributions

This course project was completed collaboratively under the supervision of Dr. Koteswar Rao Jerripothula. The team—consisting of Harsh Gour, Piyush Singal, Aditya Singh, Pulkit, and Nithin D H—operated with a shared-responsibility model. Rather than working in isolated silos, all members actively contributed to and assisted each other across every phase of the research, development, and implementation process.

Our collective efforts drove the success of the project across the following key areas:

Data Pipeline & Implementation: The team jointly managed the primary software coding in PyTorch. Together, we handled the data preparation pipeline, which included downloading, cleaning, and formatting the Kaggle medical brain datasets (the Brain CT to MRI CycleGAN dataset and the Paired CT and MRI Dataset) for model ingestion.

Architectural Design: We collaboratively designed and implemented the core architectural upgrades. This involved strategizing the integration of Convolutional Block Attention Modules (CBAM) and optimally placing channel and spatial attention gates within the ResNet bottleneck to ensure the generator focused heavily on anatomically salient structures.

Custom Loss Formulation: To prevent the baseline CycleGAN from washing out sharp anatomical boundaries, the team co-developed a custom multi-objective loss function. We replaced the standard L1 penalty with a weighted composite of LSSIM, LL1, Gradient Difference Loss (LGDL) via Sobel filters, and a novel Frequency Domain Loss (FDL/LFFT) designed by the team to partition the 2-D FFT magnitude spectrum and explicitly penalize bone boundary errors.

Computational Optimization: All members contributed to the engineering of the architecture and training pipeline to optimize it for resource-constrained GPUs. By collaboratively implementing techniques like PyTorch AMP and efficient tensor management, the team ensured the model could be trained and deployed effectively without sacrificing clinical accuracy.